

Package ‘JumpForests’

March 5, 2026

Type Package

Title Adaptive Conditional Aalen-Johansen Estimation

Version 1.0

Date 2026-03-05

Maintainer Martin Bladt <martinbladt@math.ku.dk>

Description Provides random forest versions of the Nelson-Aalen and Aalen-Johansen estimators. The methods are based on Bladt & Lemvig (2025+), in preparation.

License GPL-2 | file LICENSE

Imports Rcpp, stats, utils

LinkingTo Rcpp

SystemRequirements C++17, OpenMP

RoxygenNote 7.3.2

Encoding UTF-8

NeedsCompilation yes

Author Martin Bladt [aut, cre],
Rasmus Lemvig [aut]

Contents

JumpForests-package	2
aalen_johansen	2
jfforest	3
jfforest.error	4
jfforest.predict	4
jfforest.vimp	5
jftree	5
jftree.error	6
jftree.predict	7
prodint	7
sim_path	8
Index	9

JumpForests-package *JumpForests: Adaptive Conditional Aalen-Johansen Estimation*

Description

Provides random forest versions of the Nelson-Aalen and Aalen-Johansen estimators. The methods are based on Bladt & Lemvig (2025+), in preparation.

Author(s)

Maintainer: Martin Bladt <martinbladt@math.ku.dk>

Authors:

- Rasmus Lemvig <rfl@math.ku.dk>

aalen_johansen *Compute the conditional Aalen-Johansen estimator.*

Description

Compute the conditional Aalen-Johansen estimator.

Usage

```
aalen_johansen(
  data,
  x = NULL,
  a = NULL,
  p = NULL,
  alpha = 0.05,
  collapse = FALSE,
  weights = NULL
)
```

Arguments

data	A list of trajectory data for each individual.
x	A numeric value for conditioning.
a	A bandwidth. Defaults to Sheather and Jones selection wrt marginal.
p	An integer representing the number of states. The absorbing state is last.
alpha	Significance level (currently unused).
collapse	Logical, whether to collapse the last state of the model.
weights	A vector of observation weights.

Value

A list containing the Aalen-Johansen estimator, the Nelson-Aalen estimator, and related quantities.

jfforest*Fit a JumpForests forest*

Description

Fits a random forest for regression, survival, or multi-state data.

Usage

```
jfforest(  
  formula,  
  data,  
  feature_data = NULL,  
  splitrule = NULL,  
  mtry = NULL,  
  min_node_size = NULL,  
  nsplits = 10,  
  ntrees = NULL,  
  honest = FALSE,  
  swr = FALSE,  
  sample_rate = NULL,  
  double_bootstrap = FALSE,  
  seed = NULL,  
  nworkers = 0  
)
```

Arguments

formula	Model formula.
data	Training data.
feature_data	Optional feature data for multi-state models.
splitrule	Splitting rule.
mtry	Number of candidate features at each split.
min_node_size	Minimal node size.
nsplits	Number of split points per feature.
ntrees	Number of trees.
honest	Whether to use honest splitting.
swr	Whether to sample with replacement.
sample_rate	Sampling rate.
double_bootstrap	Whether to use double bootstrap.
seed	Optional random seed.
nworkers	Number of worker threads.

Value

A fitted forest object as a list.

jfforest.error	<i>Compute forest prediction error</i>
----------------	--

Description

Compute forest prediction error

Usage

```
jfforest.error(forest_list, new_data = NULL)
```

Arguments

forest_list	A fitted forest object from [jfforest()].
new_data	Optional evaluation data.

Value

Error metrics as a list.

jfforest.predict	<i>Predict from a fitted forest</i>
------------------	-------------------------------------

Description

Predict from a fitted forest

Usage

```
jfforest.predict(forest_list, new_data = NULL)
```

Arguments

forest_list	A fitted forest object from [jfforest()].
new_data	Optional new data.

Value

Predictions.

jfforest.vimp	<i>Variable importance for a fitted forest</i>
---------------	--

Description

Variable importance for a fitted forest

Usage

```
jfforest.vimp(forest_list, feature = NULL, seed = NULL, method = "permute")
```

Arguments

forest_list	A fitted forest object from [jfforest()].
feature	Optional feature name.
seed	Optional random seed.
method	Importance method, "permute" or "random".

Value

Variable importance values.

jftree	<i>Fit a JumpForests tree</i>
--------	-------------------------------

Description

Fits a single regression, survival, or multi-state tree.

Usage

```
jftree(  
  formula,  
  data,  
  feature_data = NULL,  
  splitrule = NULL,  
  mtry = NULL,  
  min_node_size = NULL,  
  nsplits = 10,  
  honest = FALSE,  
  seed = NULL  
)
```

Arguments

formula	Model formula.
data	Training data.
feature_data	Optional feature data for multi-state models.
splitrule	Splitting rule.
mtry	Number of candidate features at each split.
min_node_size	Minimal node size.
nsplits	Number of split points per feature.
honest	Whether to use honest splitting.
seed	Optional random seed.

Value

A fitted tree object as a list.

jftree.error	<i>Compute tree prediction error</i>
--------------	--------------------------------------

Description

Compute tree prediction error

Usage

```
jftree.error(tree_list, new_data = NULL)
```

Arguments

tree_list	A fitted tree object from [jftree()].
new_data	Optional evaluation data.

Value

Error metrics as a list.

jftree.predict	<i>Predict from a fitted tree</i>
----------------	-----------------------------------

Description

Predict from a fitted tree

Usage

```
jftree.predict(tree_list, new_data = NULL)
```

Arguments

tree_list	A fitted tree object from [jftree()].
new_data	Optional new data.

Value

Predictions.

prodint	<i>Calculate the product integral of a matrix function</i>
---------	--

Description

Calculate the product integral of a matrix function

Usage

```
prodint(start, end, step_size, lambda)
```

Arguments

start	Start time.
end	End time.
step_size	Step size of the grid.
lambda	A given matrix function.

Value

The product integral of the given matrix function.

sim_path	<i>Simulate the path of a time-inhomogeneous (semi-)Markov process until a maximal time</i>
----------	---

Description

Simulate the path of a time-inhomogeneous (semi-)Markov process until a maximal time

Usage

```
sim_path(i, rates, dists, t = 0, u = 0, tn = Inf, abs = numeric(0), bs = NA)
```

Arguments

i	The initial state, integer.
rates	The total transition rates out of states, a function with arguments state (integer), time (numeric), and duration (numeric) returning a rate (numeric).
dists	The distribution of marks, a function with arguments state (integer), time (numeric), and duration (numeric) returning a probability vector.
t	The initial time, numeric.
u	The initial duration (since the last transition), numeric. By default equal to zero
tn	The maximal time, numeric. By default equal to infinity
abs	Vector indicating which states are absorbing. By default the last state is absorbing.
bs	Vector of upper bounds on the total transition rates. By default the bounds are determined using optimize, which might only identify a local maximum.

Value

A list concerning jump times and states, with the first time being the initial time t and state and the last time being tn (if not absorbed)

Examples

```
jump_rate <- function(i, t, u){if(i == 1){3*t} else if(i == 2){5*t} else{0}}
mark_dist <- function(i, s, v){if(i == 1){c(0, 1/3, 2/3)} else if(i == 2){c(1/5, 0, 4/5)} else{0}}
sim <- sim_path(sample(1:2, 1), t = 0, tn = 2, rates = jump_rate, dists = mark_dist)
sim
```

Index

`aalen_johansen`, [2](#)

`jfforest`, [3](#)
`jfforest.error`, [4](#)
`jfforest.predict`, [4](#)
`jfforest.vimp`, [5](#)
`jftree`, [5](#)
`jftree.error`, [6](#)
`jftree.predict`, [7](#)
`JumpForests` (`JumpForests-package`), [2](#)
`JumpForests-package`, [2](#)

`prodint`, [7](#)

`sim_path`, [8](#)